

Lecture 13

Elementary Graph Algorithms



Slides are based on the textbook and its notes

Overview

- ❑ Graph problems pervade computer science, and algorithms for working with them are fundamental to the field.
- ❑ Hundreds of interesting computational problems are couched in terms of graphs.
- ❑ This lecture shows how to represent a graph in a computer and then discusses algorithms based on searching a graph using either breath-first search or depth-first search.

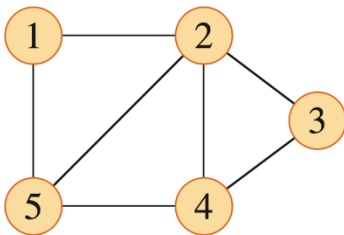
Graph representation

- Given graph $G = (V, E)$. In pseudocode, represent vertex set by $G.V$ and edge set by $G.E$.
 - G may be either directed or undirected.
 - Two common ways to represent graphs for algorithms:
 - Adjacency lists.
 - Adjacency matrix.
- When expressing the running time of an algorithm, it is often in terms of both $|V|$ and $|E|$. In asymptotic notation – and *only* in asymptotic notation – we will drop the cardinality.

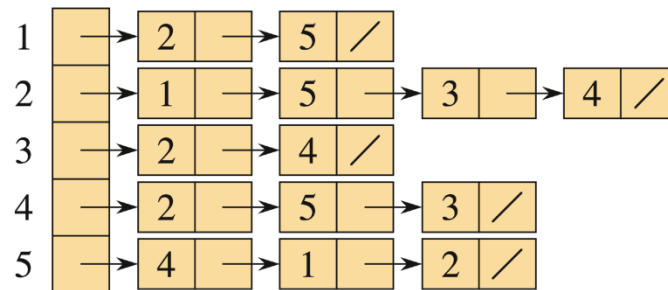
Example: $O(V + E)$ really means $O(|V| + |E|)$.

Adjacency lists and matrix

- Adjacency lists:** Array Adj of $|V|$ lists, one per vertex. Vertex u 's list has all vertices v such that $(u, v) \in E$. (Works for both directed and undirected graphs.) In pseudocode, denote the array as attribute $G.Adj$, so will see notation such as $G.Adj[u]$.
- Adjacency matrix:** $|V| \times |V|$ matrix $A = (a_{ij})$, where $a_{ij} = \begin{cases} 1 & \text{if } (i, j) \in E \\ 0 & \text{otherwise} \end{cases}$.
- Example:** (a) An undirected graph; (b) Its adjacency lists; (c) Its adjacency matrix.



(a)



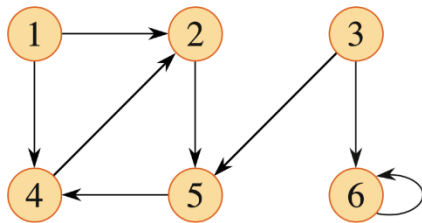
(b)

	1	2	3	4	5
1	0	1	0	0	1
2	1	0	1	1	1
3	0	1	0	1	0
4	0	1	1	0	1
5	1	1	0	1	0

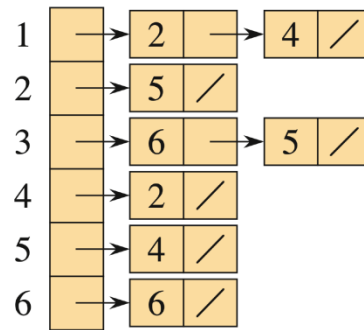
(c)

Adjacency lists and matrix

- **Example:** (a) A directed graph; (b) Its adjacency lists; (c) Its adjacency matrix.



(a)



(b)

	1	2	3	4	5	6
1	0	1	0	1	0	0
2	0	0	0	0	1	0
3	0	0	0	0	1	1
4	0	1	0	0	0	0
5	0	0	0	1	0	0
6	0	0	0	0	0	1

(c)

- Adjacency lists: If edges have **weights**, can put the weights in the lists.
- **Space:** $\Theta(V + E)$.
 - **Time:** to list all vertices adjacent to u : $\Theta(\text{degree}(u))$; to determine whether $(u, v) \in E$: $O(\text{degree}(u))$.
- Adjacency matrix: Can store weights instead of bits for weighted graph.
- **Space:** $\Theta(V^2)$
 - **Time:** to list all vertices adjacent to u : $\Theta(V)$; to determine whether $(u, v) \in E$: $\Theta(1)$.

Representing graph attributes

- ❑ Graph algorithms usually need to maintain attributes for vertices and/or edges.
- ❑ Use the usual dot-notation: denote attribute d of vertex v by $v.d$.
- ❑ Use the dot-notation for edges, too: denote attribute f of edge (u, v) by $(u, v).f$.

- ❑ *Implementing graph attributes:*
 - No one best way to implement. Depends on the programming language, the algorithm, and how the rest of the program interacts with the graph.
 - If representing the graph with adjacency lists, can represent vertex attributes in additional arrays that parallel the Adj array, e.g., $d[1 : |V|]$, so that if vertices adjacent to u are in $Adj[u]$, store $u.d$ in array entry $d[u]$.
 - But can represent attributes in other ways. Example: represent vertex attributes as instance variables within a subclass of a `Vertex` class.

Breadth-first search

- ❑ **Input:** Graph $G = (V, E)$, either directed or undirected, and a distinguished *source vertex* $s \in V$.
- ❑ **Output:**
 - $v.d$ = distance (smallest number of edges) from s to v , for all $v \in V$.
 - $v.\pi$ is v 's *predecessor* on a shortest path (smallest number of edges) from s . (u, v) is the last edge on shortest path s to v .
 - *Predecessor subgraph* contains edges (u, v) such that $v.\pi = u$. The predecessor subgraph forms a tree, called the *breath-first tree*.

Breadth-first search

Intuition

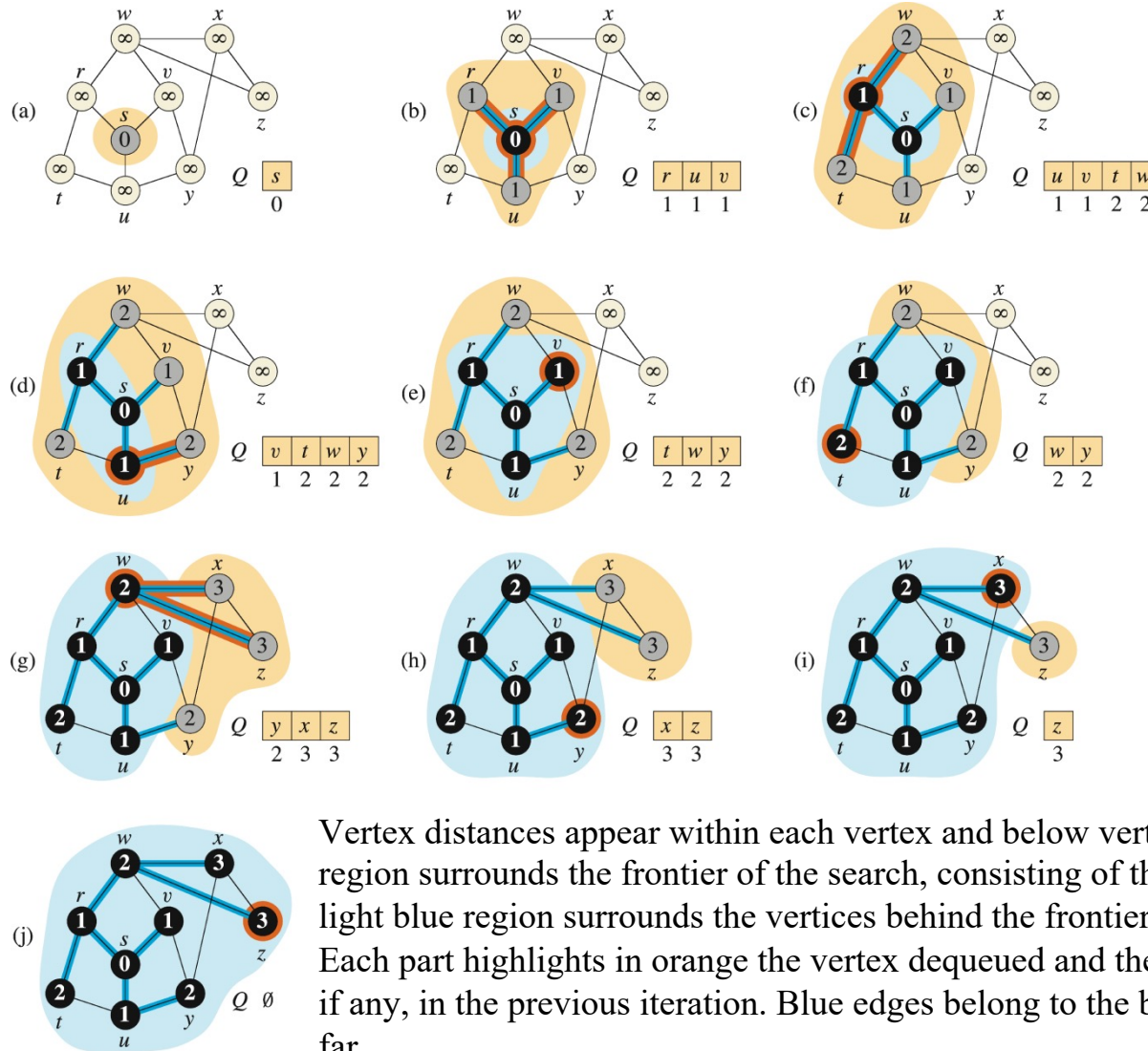
- ❑ Breath-first search expands the frontier between discovered and undiscovered vertices uniformly across the breadth of the frontier.
- ❑ Discovers vertices in waves, starting from s .
 - First visits all vertices 1 edge from s .
 - From there, visits all vertices 2 edges from s , etc.
 - Until it has discovered every vertex reachable from s .
- ❑ Use FIFO queue Q to maintain wavefront.
 - $v \in Q$ if and only if wave has visited v but has not come out of v yet.
 - Q contains vertices at a distance k , and possibly some vertices at a distance $k + 1$. Therefore, at any time Q contains portions of two consecutive waves.

Breadth-first search: Pseudocode

BFS(G, s)

```
1  for each vertex  $u \in G.V - \{s\}$ 
2       $u.color = \text{WHITE}$ 
3       $u.d = \infty$ 
4       $u.\pi = \text{NIL}$ 
5   $s.color = \text{GRAY}$ 
6   $s.d = 0$ 
7   $s.\pi = \text{NIL}$ 
8   $Q = \emptyset$ 
9  ENQUEUE( $Q, s$ )
10 while  $Q \neq \emptyset$ 
11      $u = \text{DEQUEUE}(Q)$ 
12     for each vertex  $v$  in  $G.Adj[u]$  // search the neighbors of  $u$ 
13         if  $v.color == \text{WHITE}$  // is  $v$  being discovered now?
14              $v.color = \text{GRAY}$ 
15              $v.d = u.d + 1$ 
16              $v.\pi = u$ 
17             ENQUEUE( $Q, v$ ) //  $v$  is now on the frontier
18      $u.color = \text{BLACK}$  //  $u$  is now behind the frontier
```

Breadth-first search: Example



The operation of BFS on an undirected graph. Each part shows the graph and the queue Q at the beginning of each iteration of the **while** loop of lines 10-18.

Vertex distances appear within each vertex and below vertices in the queue. The tan region surrounds the frontier of the search, consisting of the vertices in the queue. The light blue region surrounds the vertices behind the frontier, which have been dequeued. Each part highlights in orange the vertex dequeued and the breath-first tree edges added, if any, in the previous iteration. Blue edges belong to the breath-first tree constructed so far.

Breadth-first search

- Since each vertex gets a finite d value at most once, values assigned to vertices are monotonically increasing over time.
- Time = $O(V + E)$.
 - $O(V)$ because every vertex enqueued at most once.
 - $O(E)$ because every vertex dequeued at most once and edge (u, v) is examined only when u is dequeued. Therefore, every edge examined at most once if directed, at most twice if undirected.
- To print the vertices on a shortest path from s to v :

PRINT-PATH(G, s, v)

```
1  if  $v == s$ 
2      print  $s$ 
3  elseif  $v.\pi == \text{NIL}$ 
4      print “no path from”  $s$  “to”  $v$  “exists”
5  else PRINT-PATH( $G, s, v.\pi$ )
6      print  $v$ 
```

Depth-first search

- ❑ **Input:** Graph $G = (V, E)$, directed or undirected. No source vertex given.
- ❑ **Output:**
 - 2 *timestamps* on each vertex:
 - $v.d$ = discovery time
 - $v.f$ = finish time
 - There will be useful for other algorithms later on.
 - $v.\pi$ is v 's predecessor in the *depth-first forest* of ≥ 1 *depth-first trees*. If $u = v.\pi$, then (u, v) is a *tree edge*.

Depth-first search

- ❑ Methodically explores *every* edge.
 - Start over from different vertices as necessary.
- ❑ As soon as a vertex is discovered, explore from it.
 - Unlike BFS, which puts a vertex on a queue so that it is explored from later.
- ❑ As DFS progresses, every vertex has a *color*:
 - WHITE = undiscovered,
 - GRAY = discovered, but not finished (not done exploring from it)
 - BLACK = finished (have found everything reachable from it)
- ❑ Discovery and finish times:
 - Unique integers from 1 to $2|V|$ and for all v , $v.d < v.f$.

In other words, $1 \leq v.d < v.f \leq 2|V|$.

- ❑ Time = $\Theta(V + E)$.
 - Similar to BFS analysis.
 - Θ , not just O , since guaranteed to examine every vertex and edge.
- ❑ Each depth-first tree is made of edges (u, v) such that u is gray and v is white when (u, v) is explored.

Depth-first search: Pseudocode

- ▣ Uses a global timestamp *time*.

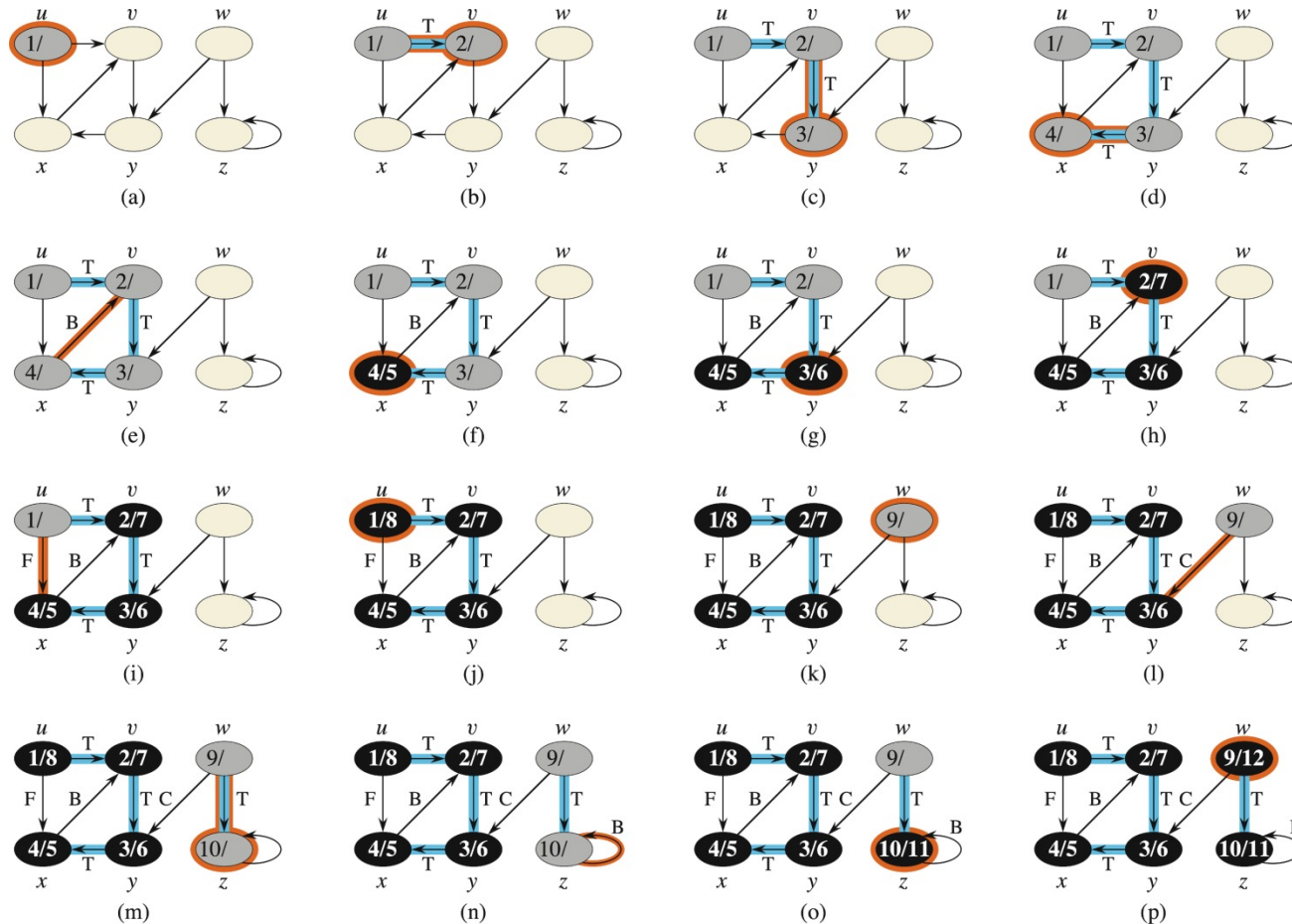
DFS(*G*)

```
1  for each vertex  $u \in G.V$ 
2       $u.color = \text{WHITE}$ 
3       $u.\pi = \text{NIL}$ 
4   $time = 0$ 
5  for each vertex  $u \in G.V$ 
6      if  $u.color == \text{WHITE}$ 
7          DFS-VISIT( $G, u$ )
```

DFS-VISIT(G, u)

```
1   $time = time + 1$                 // white vertex  $u$  has just been discovered
2   $u.d = time$ 
3   $u.color = \text{GRAY}$ 
4  for each vertex  $v$  in  $G.Adj[u]$     // explore each edge  $(u, v)$ 
5      if  $v.color == \text{WHITE}$ 
6           $v.\pi = u$ 
7          DFS-VISIT( $G, v$ )
8   $time = time + 1$ 
9   $u.f = time$ 
10  $u.color = \text{BLACK}$               // blacken  $u$ ; it is finished
```

Depth-first search: Example



Edges are classified as they are explored: tree edges are labeled T, back edges B, forward edges F, and cross edges C. Timestamps within vertices indicate discovery/finish times. Tree edges are highlighted in blue. Orange highlights indicate vertices whose discovery or finish times change and edges that are explored in each step.

Reading

- ▣ Sections 20.1 ~ 20.3.